

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Лекция – 7 (2)***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция – 7 (2)

Методы организации работы в команде разработчиков. Системы контроля версий

Управление и планирование в программных проектах

Менеджеры в организациях, создающих и сопровождающих программное обеспечение (программные менеджеры – software managers) выполняют множество функций. Они ответственны за планирование работ в проекте. Они следят за тем, чтобы работа выполнялась в соответствии с заданными стандартами. Они наблюдают за ходом работ, чтобы убедиться в том, что разработка осуществляется вовремя и в пределах бюджета. Хорошее управление не может гарантировать успех проекта. Плохое управление обычно приводит к провалу проекта. В этом случае программное средство поставляется с опозданием, стоит дороже, чем оценено первоначально, и не соответствует требованиям.

Понятия управления программными проектами

Программные менеджеры выполняют такую же работу, как и менеджеры в других инженерных проектах. Но технология программирования во многих вопросах отличается от других типов инженерных дисциплин. В частности, выделяют следующие отличия.

Продукт таких проектов является "неосязаемым".

Не существует стандартного процесса.

Большие программные проекты часто являются "особыми" проектами.

В этой главе рассматриваются основные понятия управления (менеджмента) программных проектов.

Процесс управления в стандарте ISO/IEC 12207

Процесс управления (management process) – один из организационных процессов стандарта ISO/IEC 12207.

Менеджер (manager) ответственен за управление продуктом, управление процессом и управление каждой задачей применяемого(-ых) процесса(-ов).

Процесс управления включает в себя пять основных видов деятельности:

1) инициирование процесса и определение его сферы деятельности;

2) планирование процесса;

3) выполнение процесса и управление этим процессом;

4) экспертизу и оценивание;

5) закрытие процесса.

Инициирование и определение сферы деятельности процесса (initiation and scope definition).

Планирование процесса (planning). Одна, но важная задача. Менеджер разрабатывает планы для выполнения процесса. Эти планы должны включать следующую (но могут включать и дополнительную) информацию:

а) графики для своевременного завершения задач;

б) оценки необходимых усилий;

в) оценки ресурсов, необходимых для выполнения задач;

г) распределение задач между участниками процесса;

д) исполнителей;

е) количественные оценки рисков, связанных с задачами или самими процессами;

ж) метрики управления качеством, которые должны использоваться на протяжении всего процесса;

з) стоимости, связанные с выполнением процесса;

и) обеспечение средой выполнения и инфраструктурой.

Выполнение процесса и управление им (execution and control).

Экспертиза и оценивание (review and evaluation).

Закрытие (closure).

Спектр управления

Эффективное управление программными проектами фокусирует свое внимание на трех главных компонентах: людях (people), проблеме (problem) и процессах (processes).

Люди

Модель зрелости способностей управления людьми (People Management Capability Maturity Model –PM CMM) определяет следующие ключевые области практики для людей в программном обеспечении: набор, отбор, управление производительностью, обучение, вознаграждение, развитие карьеры, организация и планирование работы, развитие бригады/культуры. Организации, которые достигают высоких уровней зрелости в области управления людьми, имеют более высокую вероятность реализации эффективной практики технологии программирования.

Проблема

До написания плана проекта должны быть установлены его цели и сфера охвата; должны быть рассмотрены альтернативные решения; должны быть идентифицированы технические и управленческие ограничения. Без этой информации невозможно определить разумные (и точные) оценки стоимости, эффективную оценку риска, реалистичное разделение проекта на задачи или управляемый график проекта, который обеспечивает понятную интерпретацию продвижения в проекте.

Процесс

Процесс разработки конкретного программного средства – это схема (framework), используемая для создания исчерпывающего плана разработки. Небольшое число видов деятельности схемы (framework activities) применимы ко всем программным проектам, независимо от размера и сложности создаваемого программного обеспечения. Множества задач (task sets) схемы, которыми могут быть собственно задачи из ISO/IEC 12207, этапы графика работ, создание поставляемых документов и проведение мероприятий обеспечения качества, дают возможность подгонять виды деятельности схемы под характеристики конкретного программного проекта и требования бригады проекта. Охватывающие виды деятельности, такие как обеспечение качества программных средств, управление конфигурацией программных средств и собственно управление, накладываются на модель процессов. Охватывающие виды деятельности независимы по отношению к видам деятельности схемы и осуществляются на протяжении всех процессов.

Виды деятельности по управлению

Большинство менеджеров имеют ответственность в некоторых или во всех следующих видах деятельности на определенных этапах проекта:

в написании предложений;

в оценке стоимости проекта;

в планировании проекта и построении графика работ;

в контроле хода проекта и экспертизах;

в подборе кадров и их оценивании;

в написании отчетов и их представлении.

Написание предложений (proposal writing).

Планирование проекта и составление графика работ (project planning and scheduling).

Контроль хода проекта и экспертизы (project monitoring and reviews).

Подбор людей для работы в проекте (personal selection and evaluation). Проблемы.

Менеджеры проекта должны писать сжатые, согласованные документы, содержащих критичную информацию из детальных отчетов проекта (report writing and presentation).

Планирование проекта

Эффективность управления программным проектом зависит от тщательности планирования хода работ в проекте. Менеджер проекта должен предвидеть возможные проблемы и найти предварительные решения этих проблем. План, подготовленный в начале проекта, должен использоваться как управляющий документ проекта. Этот первоначальный план не является догмой, он должен изменяться с ходом работ в проекте и с появлением более точной информации.

План	**Описание**
План качества	Описывает процедуры обеспечения качества и стандарты, используемые в проекте
План подтверждения	Описывает подход, ресурсы и график работ при подтверждении системы
План управления конфигурацией	Описывает процедуры управления конфигурацией и структуры, которые должны при этом использоваться
План сопровождения	Прогнозирует требования к сопровождению системы, стоимость сопровождения и необходимые усилия
План совершенствования персонала	Описывает, как будут совершенствоваться мастерство и опыт членов команды проекта

Рис. 7. Типы планов.

Установить ограничения проекта
Сделать начальную оценку параметров проекта
Определить создаваемые в проекте документы и поставляемые документы
Продолжать, пока проект не завершен или не прекращен:
    Составить график работ
    Инициировать работы в соответствии с графиком
    Ждать (некоторое время)
    Провести экспертизу хода работ в проекте
    Пересмотреть оценки параметров проекта
    Изменить график работ
    Заново обсудить ограничения проекта и поставляемые документы
    Если возникают проблемы
        Инициировать техническую экспертизу и возможный пересмотр
    Конец Если
Конец Продолжать

Рис. 8. Процесс планирование проекта.

План проекта

План проекта устанавливает ресурсы, необходимые для выполнения проекта, распределение работ и график для выполнения этих работ. Большинство планов должны включать в себя следующие разделы.

Введение.
Организация проекта.
Анализ рисков.
Требования к необходимым аппаратным и программным ресурсам.
Распределение работ.
График выполнения работ в проекте.
Механизмы контроля и отчеты.

Каждый план проекта должен регулярно пересматриваться по ходу проекта.

Организация работы

Менеджеры нуждаются в информации. Программное обеспечение неосязаемо, информация может быть представлена только в форме документов, описывающих выполненную работу.

Когда планируется проект, то устанавливается серия контрольных точек, где каждый такая точка является конечной точкой в некотором виде деятельности в программном проекте. В каждой такой контрольной точке менеджменту должен быть представлен формальный отчет о ходе работ. Эти контрольные точки должны представлять конец некоторого явно выраженного этапа в проекте.

Поставляемые (подлежащие сдаче) документы и программы (deliverables) являются результатом проекта, поставляемым заказчику. Подлежащие сдаче документы – отчетные документы (т.е., milestones). Однако, отчетные документы не обязательно подлежат поставке заказчику.

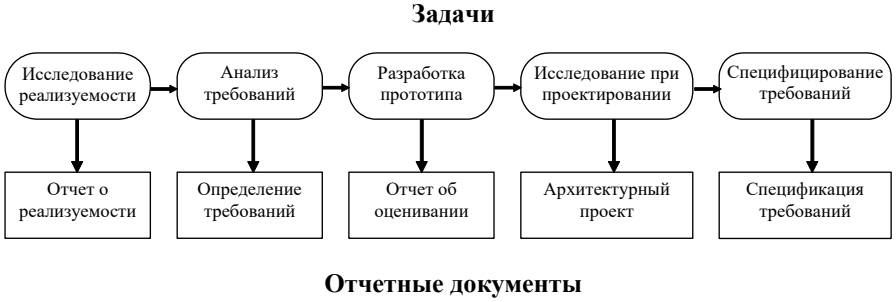


Рис. 9. Отчетные документы в процессе установления требований.

Одна из причин широкого распространения каскадной модели программного процесса – она обеспечивает прямое определение контрольных точек на протяжении всего проекта.

Управление рисками

Риски программного обеспечения

Нет общепринятого определения понятия риск программного обеспечения (software risk). Но есть два обязательных компонента риска: неопределенность (uncertainty) и потери (loss).

Ответная и профилактическая стратегии рисков

Ответная (reactive) стратегия рисков может быть представлена следующими словами: "Не следует беспокоится о проблемах, пока они не произойдут.

К сожалению, большинство бригад разработчиков программных средств надеются исключительно на ответные стратегии рисков. Чаще всего бригада программного обеспечения не делает ничего с рисками, пока что-нибудь не идет плохо. Тогда бригада впадает в "кипучую деятельность" в попытке быстро исправить положение. При этом зачастую проект находится в реальной опасности.

Значительно более разумной стратегией для управления рисками является профилактическая (proactive). Она начинает реализовываться задолго до того, как начнется техническая работа.

Идентифицируются потенциальные риски,

оцениваются их вероятности и влияние, и они располагаются по важности.

Затем бригада проекта устанавливает план управления рисками.

Идентификация рисков

Один из методов идентификации рисков заключается в создании контрольного списка рисков (a risk item checklist). Этот контрольный список может использоваться для идентификации рисков, и он фокусирует свое внимание на подмножестве известных и

предсказуемых рисков по подкатегориям.

Размер продукта

Коммерческое влияние

Характеристики заказчика

Определение процессов
Среда разработки
Технология, которая должна быть разработана
Размеры бригады и ее опыт
Компоненты и причины возникновения риска
Компоненты риска программного средства (software risk components):
риск эксплуатационных характеристик,
риск стоимости,
риск поддержки,
риск графика работы.

Разработка таблицы рисков

Риски	**Категория**	**Вероятность**	**Влияние**	**RMMM**
Оценка размера может быть существенно заниженной	PS	60	2	
Большее число пользователей, чем было запланировано	PS	30	3	
Меньше повторного использования, чем было запланировано	PS	70	2	
Конечные пользователи противодействуют системе	BU	40	3	
Срок поставки будет отодвинут	BU	50	2	
Финансирование будет потеряно	CU	40	1	
Заказчик изменит требования	PS	80	2	
Технология не будет удовлетворять ожиданиям	TE	30	1	
Недостаток обучения на готовых средствах	DE	80	3	
Члены команды не имеют достаточного опыта	ST	30	2	
Текучесть кадров будет высокой	ST	60	2	

Категория: PS - Program Size; BU -BUsiness risks; CU - CUstomer risks; TE - TEchnology risks;
DE - DEvelopment risks; ST - STuff risks.
Значения влияния: 1 – катастрофическое, 2 – критическое, 3 – предельное, 4 – несущественное.

Рис 10. Простая таблица рисков до сортировки.

Оценивание влияния риска

Три фактора влияют на последствия, которые вероятны, если риск действительно реализуется: его природа, его охват и его время.

Смягчение рисков, контроль и управление

Эффективная стратегия управления рисками должна рассматривать три вопроса:
избежание рисков;
контроль рисков и
управление рисками и планирование непредвиденных ситуаций.

Процесс управления конфигурацией

"Искусство координирования разработки программного обеспечения для минимизации ... путаницы называется управлением конфигурацией. Управление конфигурацией – искусство идентификации, организации и управления модификациями разрабатываемого программного средства командой разработчиков. Целью этих действий является максимизация продуктивности при минимизации ошибок" [Pressman-97].

Управление конфигурацией программного средства (УКПС) – охватывающая деятельность, осуществляемая на протяжении всего жизненного цикла программного средства.

Различия между сопровождением программного средства и УКПС.

Процесс Управления Конфигурацией (Configuration Management Process) – процесс применения административных и технических процедур на протяжении жизненного цикла программного средства для:

идентификации, определения и превращения в эталоны основных компонентов программного средства в системе;

управления изменениями и выпусками этих компонентов;

создания отчетов о состоянии компонентов и запросов на изменение;

обеспечения полноты, непротиворечивости и правильности этих компонентов и

управления хранением, манипулированием и поставкой компонентов.

Основные понятия управления конфигурацией программного средства

Результат программного процесса:

программы для ЭВМ (в исходном и исполняемом представлении),

документы, описывающие компьютерные программы (и предназначенные как для разработчиков, так и для пользователей), и

данные (содержащиеся в программах или являющиеся внешними по отношению к ней).

Все это вместе называется конфигурацией программного средства (software configuration).

Согласно Первому закону технологии разработки систем [Pressman-97]: Не важно, в каком месте жизненного цикла вы находитесь, система будет изменяться, и желание изменить ее будет сохраняться на протяжении всего жизненного цикла.

Система контроля версий

Система контроля версий – это обязательный инструмент в арсенале программиста любого уровня: от новичка, который только осваивается в профессии, до тимлида, чей опыт исчисляется многими годами и проектами.

И если профи хорошо представляют себе задачи и возможности такой системы, то начинающим необходимо дать некоторые объяснения. Впрочем, иногда и опытные программисты совершают ошибки при работе с системой.

В нашей статье мы расскажем, зачем нужен данный инструмент, как им правильно пользоваться и разберем некоторые ошибки, которые допускают чаще всего.

Принцип работы системы контроля версий

Пожалуй, многие представляют себе, каким образом вносятся изменения в тот или иной проект в процессе работы. То есть, тут стоит задача не потерять существующую работоспособную версию проекта. Для этого обычно создается новая папка (чаще всего ее так и называют «Новая папка», дополняя имя, к примеру, датой или парой символов), в которую вы копируете имеющуюся версию и все дальнейшие изменения производите уже именно с ней.

Подобных папок может набраться довольно много. Из-за этого в какой-то момент усложняется откат к предыдущим версиям, труднее становится контролировать и упорядочивать вносимые изменения и т.д. А если проект ведут несколько специалистов, картина усугубляется.

Принцип работы системы контроля версий

Системы контроля версий (СКВ или VCS) разработаны специально для того, чтобы максимально упростить и упорядочить работу над проектом (вне зависимости от того, сколько человек в этом участвуют). СКВ дает возможность видеть, кто, когда и какие изменения вносил; позволяет формировать новые ветви проекта, объединять уже имеющиеся; настраивать контроль доступа к проекту; осуществлять откат до предыдущих версий.

Задачи для системы контроля версий

В целом задачи для системы контроля версий уже описаны выше, но можно пояснить еще, для тех, кто сталкивается с термином впервые. Для наглядности представьте, что работа над проектом – это игра, которую нужно пройти, двигаясь от одной контрольной точки – к другой (это и будут промежуточные версии проекта).

Но чего можно ожидать, если программист на своем компьютере не использовал систему контроля версий и не сохранял создаваемые промежуточные копии плагина? Специалист

будет переделывать проект практически заново. Возможно, ему и удастся внести нужные изменения, не нарушив основную логику, но это будет очень и очень сложно.

А теперь представьте, что работу над плагином ведут несколько человек. Тогда применение СКВ даже не обсуждается. Это позволит каждому разработчику, отделившись от основной ветки, выполнять свой конкретный круг задач. После чего специалисты проверят код на ошибки и объединят вместе все части проекта.

Работа в системе контроля версий дает возможность сохранять промежуточные варианты проекта, плюс это еще и помогает справляться с проблемами в случае их возникновения. Когда, например, несколько программистов, работающих над одной опцией, одновременно сбросят в репозиторий созданные версии, то система в автоматическом режиме займется устранением конфликта.

Если система не справится с проблемой, то разработчики получат информацию об этом и вручную всё «подчистят». В конечном счете, именно программист контролирует и процесс, и окончательный результат.

Вот перечень задач, выполняемых системой контроля версий файлов:

- Сохранение исходного кода. Информация сваливается на удаленный сервер и в репозитории остаются даже файлы, удаленные с компьютера разработчика.
- Возможность привлекать группу программистов, не покупая отдельно специальные инструменты для командной работы. Каждый свою задачу на персональном компьютере, обновляя файлы, когда это нужно.
- Отмена внесенных изменений. Всегда есть возможность вернуться к контрольной точке, провести ревью исходного кода и текущего, а затем обновить основную ветку.
- Распределенная работа над проектом. То есть, программисты могут создавать видоизмененный плагин, пока основная его версия спокойно функционирует на сайте.

Типы систем контроля версий

Локальные СКВ

Среди первых систем подобного плана была RCS, разработанная в 1985 году и сразу ставшая популярной.

Данная система контроля версий хранит и все зарегистрированные в ней файлы, и любые вносимые в них изменения. Специально к текстовым файлам применяется алгоритм дельта-компрессии. Это означает, что система сохраняет последнюю версию и все предшествующие ей изменения.

СКВ подобного плана решали лишь проблему хранения больших объемов данных.

Современные СКВ условно бывают двух видов, а именно, централизованные и распределенные системы контроля версий.

Централизованные СКВ

Далее предстояло решить проблему привлечения к работе над одним проектом нескольких специалистов (за разными компьютерами). Для этого и были разработаны централизованные системы контроля версий, такие как, к примеру, CVS, Subversion, Perforce. Они имеют собственный центральный сервер, где накапливаются все файлы проекта. Система контролирует и их сохранность, и выдачу их копий определенному ряду пользователей. Именно по такой схеме много лет и работали СКВ.

Централизованные СКВ

В целом подход хороший. Он позволяет администраторам достаточно просто отслеживать действия каждого привлеченного разработчика. Если сравнивать с локальными базами по каждому клиенту, то там осуществлять контроль куда сложнее.

Впрочем, и минусы тут, несомненно, есть. Главный из них – уязвимость централизованного сервера. Временное выключение сервера (пусть даже на час) останавливает работу программистов, они не могут ни сохранять новые варианты версий, ни взаимодействовать между собой. В случае же повреждения диска, на котором хранится центральная база данных, все наработки по проекту теряются безвозвратно.

Распределенные СКВ

И вот тут незаменимыми становятся распределенные системы контроля версий. Это, к примеру, Git, Mercurial, Bazaar, Darcs. Суть их работы состоит в выгрузке клиентам не только версий файлов с последними изменениями, а всего репозитория. Репозиторий (на сленге – «репа») – это место хранения и поддержания данных. Более того, условно говоря, различают центральный и локальные репозитории.

Из локальных изменения выгружаются в центральный, с которым они (локальные репозитории) непременно синхронизируются. И в случае «обвала» основного сервера продолжить работу можно, взяв базу данных с любого клиентского компьютера.

Потому что как только клиент скачивает измененную версию файлов, к нему в компьютер копируются все данные репозитория в полном объеме.

Еще один плюс подобных систем в том, что в большинстве из них доступно для хранения данных сразу нескольких удаленных репозиториев.

Сегодня практически стандартом по умолчание стало использование распределенной системы контроля версий GIT. Впрочем, популярные в прошлом варианты вроде Subversion тоже можно еще встретить в старых масштабных проектах.

Обзор наиболее популярной системы контроля версий GIT

Современный программист вне зависимости от его профиля должен знать, как работать с GIT. Да, у разных СКВ есть свои достоинства и недостатки, но большая часть компаний на сегодняшний день используют именно систему контроля версий GIT.

GIT относится к распределенным системам контроля версий, и как децентрализованная СКВ имеет свои преимущества:

- сетевых задержек не бывает, поэтому все операции выполняются очень быстро;
- идеально подходит для проектов, к работе над которыми привлекается целая команда специалистов;
- возможность сохранить данные в случае выхода из строя центрального сервера.

Версии хранятся тут в двух репозиториях, локальном (расположен на компьютере разработчика, причем именно репозиторий с полным объемом данных, а не ссылки на отдельные версии проекта) и удаленном (лежит на удаленном сервере). Получение данных с удаленного репозитория идет через гит-хостинги Github, Google Code, GitLab и прочее.

Достаточно лишь иметь интернет соединение, чтобы получать доступ к удаленному репозиторию. Взаимодействие просто представляет собой синхронизацию локального и удаленного репозиториев.

Копирование данных из локального репозитория в удаленный в данной системе контроля версий осуществляется командой push, а обратный процесс – командой pull.

Любые изменения в документе и его новые версии сохраняются в локальном репозитории.

Набор всех сохраненных версий называют «деревом» проекта, и оно находится в репозитории.

Бывает прямое дерево, когда сохранения файлов выполняются последовательно, одно за другим без возврата к предыдущим вариантам.

Есть еще разветвленное дерево. Оно получается, если приходится дорабатывать старые версии, вносить в них изменения и потом сохранять.

Любые сохранения, получившиеся на ветках дерева, изначально наращиваются на один файл, называемый исходным. В процессе работы в него вносятся изменения. Система управления версиями позволяет одновременно корректировать, дополнять

любую из веток дерева. В итоге некоторые из них «срастаются». Все изменения обязательно каждый раз сохраняются в последней версии файла.

Обзор наиболее популярной системы контроля версий GIT

Для настройки и последующей работы с системой контроля версий GIT понадобится регистрация на каком-либо из git-хостингов, к примеру на Github, Sourceforge, Google Code, GitLab, Codebase или на других.

Наибольшей популярностью из названых пользуется Github.

Почему именно он? Благодаря удобному интерфейсу, открытому коду (то есть, платить за размещение проекта на хостинге не нужно). Плюс у Github довольно широкие функциональные возможности.

В распоряжении пользователя тут целый набор графических git-клиентов, то есть специальных программ, позволяющих задействовать графический интерфейс для работы над проектом.

Существует много IDE, допускающих использование GIT.

Список терминов для работы с системой контроля версий GIT

Git (Гит) – система контроля версий файлов.

GitHub (Гитхаб) – специальный сервис, на котором хранятся репозитории. Именно через него идет работа над проектами.

Репозиторий GIT – файловый каталог, то есть, место, где лежат: файлы конфигураций; файлы с журналами проведенных в репозитории операций; индекс расположения файлов; и, собственно, само хранилище контролируемых файлов.

Локальный репозиторий – размещается в компьютере разработчика в каталоге. Все изменения проводятся и сохраняются сначала здесь, а потом уже пересылаются на удаленный репозиторий.

Удаленный репозиторий – тот, что размещен на удаленном сервере. Тут собираются все изменения, внесенные в проект, и здесь же их можно взять при необходимости.

Fork (Форк) – это копия репозитория, которая, по сути, представляет собой внешнюю ветку текущего репозитория. Такую копию может создать на Гитхабе любой пользователь, поработать с ней и через пулреквест прислать на ваш репозиторий измененный вариант.

Обновиться из апстрима – это когда своя локальная версия форка обновляется до последней сохраненной в репозитории версии (с которой и создавался форк).

Обновиться из ориджина – это когда версия локального репозитория обновляется до версии соответствующего ему удаленного репозитория.

Clone (Клонирование) – создание отдельного каталога, в который скачивается репозиторий с удаленного сервера. Далее этот каталог используется в работе в качестве репозитория.

Branch (Ветка) – параллельное «ответвление» в репозитории. С этой составляющей можно отдельно работать, не «задевая» при этом главную версию (с которой ветку можно вновь объединить, после проведения необходимых изменений).

Master (Мастер) – основная ветка репозитория, называемая еще главной.

Commit (Коммит) – сохранение изменений в репозиторий. Выполняется разработчиком на своем локальном компьютере.